

Problem Description 1

Implement a CNN feature-extraction pipeline for a grayscale image. Given an input image, a convolution kernel, and a bias, generate the resulting feature map.

Use the following conditions:

- $\text{Stride} = 1$
- $\text{Padding} = 0$
- Do not flip the kernel; use the CNN cross-correlation convention.
- Add the supplied bias to every convolution response.
- Apply ReLU activation to the resulting values.

For an input image of size $H \times W$ and kernel of size $K_H \times K_W$, calculate the output dimensions as:

$$\begin{aligned} O_H &= H - K_H + 1 \\ O_W &= W - K_W + 1 \end{aligned}$$

For every valid image patch:

```
response = sum(patch * kernel)
```

Finally:

```
output = ReLU(response + bias)
```

where:

$$\text{ReLU}(x) = \max(0, x)$$

Complete the functions provided in the boilerplate to generate the final feature map.

Input Format

$H \ W$

```
image row 1
image row 2
...
image row H
```

$K_H \ K_W$

```
kernel row 1
kernel row 2
...
kernel row  $K_H$ 
```

bias

Output Format

First print the dimensions of the final feature map:

$O_H \ O_W$

Then print the feature map, one row per line.

Constraints

```
2 <= H, W <= 20
1 <= KH <= H
1 <= KW <= W
```

Image, kernel, and bias values are integers.

Stride is always 1.

Padding is always 0.

Kernel flipping must not be performed.

Boilerplate

HEAD

```
import sys
import numpy as np
```

BODY

```
def cnn_feature_map(image, kernel, bias):
    # Write your code here using NumPy
    pass
```

TAIL

```
data = list(map(int, sys.stdin.read().strip().split()))
it = iter(data)
```

```
H = next(it)
W = next(it)
```

```
image = np.array(
    [
        [next(it) for _ in range(W)]
        for _ in range(H)
    ],
    dtype=int
)
```

```
KH = next(it)
KW = next(it)
```

```
kernel = np.array(
    [
        [next(it) for _ in range(KW)]
        for _ in range(KH)
    ],
    dtype=int
)
```

```
bias = next(it)
```

```
result = cnn_feature_map(
    image,
    kernel,
    bias
)
```

```
OH, OW = result.shape
```

```
print(OH, OW)

for row in result:
    print(*row)
```

Test Case 1

Input

```
3 3
1 2 3
4 5 6
7 8 9
2 2
1 0
0 -1
5
```

Output

```
2 2
1 1
1 1
```

Problem Description 2

A transformer processes a sequence of n tokens, where each token is represented by d features. Implement **causal scaled dot-product self-attention from scratch using NumPy**.

You are given an input matrix X and three projection matrices W_Q , W_K , and W_V .

Calculate:

```
Q = X @ W_Q
K = X @ W_K
V = X @ W_V
```

The attention scores are:

```
scores = (Q @ K.T) / sqrt(d)
```

The attention mechanism is causal.

A token at position i may attend only to itself and tokens appearing before it.

Any token appearing after position i must therefore receive an attention probability of 0.

Convert the permitted attention scores into probabilities using row-wise Softmax.

The Softmax calculation must remain numerically stable even when attention scores are large.

Calculate the contextual representation as:

```
contextual_output = attention_weights @ V
```

Your program must output both:

- the complete attention-weight matrix
- the contextual representation matrix

Do not use PyTorch, TensorFlow, Hugging Face attention layers, or any built-in attention implementation.

Input Format

$n \ d$

X row 1

...

X row n

WQ row 1

...

WQ row d

WK row 1

...

WK row d

WV row 1

...

WV row d

Where:

$X = n \times d$

$WQ = d \times d$

$WK = d \times d$

$WV = d \times d$

Output Format

Print the $n \times n$ attention-weight matrix first.

Then print the $n \times d$ contextual representation matrix.

Print every value to exactly four decimal places.

Do not print labels or additional text.

Constraints

$1 \leq n \leq 10$

$1 \leq d \leq 10$

Matrix elements may be positive, negative, zero, or decimal values.

Scaling must use $\sqrt{d}$.

Softmax must operate independently on each row.

Future-token attention probabilities must be 0.

The implementation must handle large attention scores without numerical overflow.

Boilerplate

HEAD

```
import sys
import numpy as np
```

BODY

```
def causal_self_attention(
    X,
    WQ,
    WK,
    WV
):
    # Write your code here using NumPy
    pass
```

TAIL

```
data = sys.stdin.read().strip().split()
it = iter(data)

n = int(next(it))
d = int(next(it))

X = np.array(
    [
        [float(next(it)) for _ in range(d)]
        for _ in range(n)
    ],
    dtype=float
)

WQ = np.array(
    [
        [float(next(it)) for _ in range(d)]
        for _ in range(d)
    ],
    dtype=float
)

WK = np.array(
    [
        [float(next(it)) for _ in range(d)]
        for _ in range(d)
    ],
    dtype=float
)

WV = np.array(
    [
        [float(next(it)) for _ in range(d)]
        for _ in range(d)
    ],
    dtype=float
)

attention_weights, contextual_output = causal_self_attention(
    X,
    WQ,
    WK,
    WV
)

for row in attention_weights:
    print(
```

```

        " ".join(
            f"{value:.4f}"
            for value in row
        )
    )

for row in contextual_output:
    print(
        " ".join(
            f"{value:.4f}"
            for value in row
        )
    )

```

Test Case 1

Input

```

2 2
1 0
0 1
1 0
0 1
1 0
0 1
1 0
0 1
1 0
0 1

```

Output

```

1.0000 0.0000
0.3302 0.6698
1.0000 0.0000
0.3302 0.6698

```

Problem Description 3

Implement a max-pooling operation for a CNN feature map using NumPy.

You are given:

- a two-dimensional feature map
- a square pooling-window size
- a stride value

For every valid pooling window, select the maximum value present in that window.

No padding is applied.

For a feature map of size $H \times W$, pooling window of size $P \times P$, and stride S , calculate:

$$OH = ((H - P) // S) + 1$$

$$OW = ((W - P) // S) + 1$$

For every output position:

```
window = corresponding P x P region  
output_value = maximum value in window
```

Only complete pooling windows must be processed.

Use NumPy operations to extract each window and determine its maximum value.

Input Format

```
H W  
P S  
feature_map row 1  
feature_map row 2  
...  
feature_map row H
```

Output Format

First print the output dimensions:

```
OH OW
```

Then print the pooled feature map, one row per line.

Constraints

```
1 <= H, W <= 20  
1 <= P <= min(H, W)  
1 <= S <= 10  
Feature-map values are integers.  
No padding is used.
```

Boilerplate

HEAD

```
import sys  
import numpy as np
```

BODY

```
def max_pool(feature_map, pool_size, stride):  
    # Write your code here using NumPy  
    pass
```

TAIL

```
data = list(map(int, sys.stdin.read().strip().split()))  
it = iter(data)  
  
H = next(it)  
W = next(it)  
  
P = next(it)  
S = next(it)  
  
feature_map = np.array(  
    [  

```

```

        [next(it) for _ in range(W)]
        for _ in range(H)
    ]
)

result = max_pool(feature_map, P, S)

OH, OW = result.shape

print(OH, OW)

for row in result:
    print(*row)

```

Test Case 1

Input

```

4 4
2 2
1 3 2 0
4 6 5 1
0 2 7 8
1 3 2 4

```

Output

```

2 2
6 5
3 8

```

Problem Description 4

Implement one complete **mini-batch training iteration** of a two-layer neural network for binary classification using NumPy.

The network receives m training samples, each containing d features, and contains h hidden neurons.

The hidden layer uses ReLU activation, while the output layer uses Sigmoid activation.

Training uses the mean Binary Cross-Entropy loss over the batch.

For all samples:

1. Perform forward propagation.
2. Calculate the mean BCE loss.
3. Calculate gradients for w_1 , b_1 , w_2 , and b_2 .
4. Update all parameters using the supplied learning rate.
5. Perform another forward pass using the updated parameters.

For ReLU, use derivative:

```

1 when pre-activation > 0
0 when pre-activation <= 0

```

Return:

- mean loss before the update
- updated w_1
- updated b_1
- updated w_2
- updated b_2
- probabilities after the update
- predicted classes after the update

A probability greater than or equal to 0.5 belongs to class 1.

Do not use automatic differentiation or deep-learning libraries.

Input Format

m d h

X row 1

...

X row m

w_1 row 1

...

w_1 row h

b_1

w_2

b_2

y_1 y_2 ... y_m

learning_rate

w_1 has dimensions $h \times d$.

b_1 contains h values.

w_2 contains h values.

b_2 is a scalar.

Output Format

mean_loss

updated w_1 rows

updated b_1

updated w_2

updated b_2

updated probabilities

predicted classes

Print all floating-point values to exactly four decimal places.

Boilerplate

HEAD

```
import sys
import numpy as np
```

BODY

```
def train_one_batch(
    X,
    W1,
    b1,
    W2,
    b2,
    y,
    learning_rate
):
    # Write your code here using NumPy
    pass
```

TAIL

```
data = sys.stdin.read().strip().split()
it = iter(data)

m = int(next(it))
d = int(next(it))
h = int(next(it))

X = np.array(
    [
        [float(next(it)) for _ in range(d)]
        for _ in range(m)
    ],
    dtype=float
)

W1 = np.array(
    [
        [float(next(it)) for _ in range(d)]
        for _ in range(h)
    ],
    dtype=float
)

b1 = np.array(
    [float(next(it)) for _ in range(h)],
    dtype=float
)

W2 = np.array(
    [float(next(it)) for _ in range(h)],
    dtype=float
)

b2 = float(next(it))

y = np.array(
    [float(next(it)) for _ in range(m)],
    dtype=float
```

```

)

learning_rate = float(next(it))

(
    mean_loss,
    updated_W1,
    updated_b1,
    updated_W2,
    updated_b2,
    probabilities,
    predictions
) = train_one_batch(
    X,
    W1,
    b1,
    W2,
    b2,
    Y,
    learning_rate
)

print(f"{mean_loss:.4f}")

for row in updated_W1:
    print(
        " ".join(
            f"{value:.4f}"
            for value in row
        )
    )

print(
    " ".join(
        f"{value:.4f}"
        for value in updated_b1
    )
)

print(
    " ".join(
        f"{value:.4f}"
        for value in updated_W2
    )
)

print(f"{updated_b2:.4f}")

print(
    " ".join(
        f"{value:.4f}"
        for value in probabilities
    )
)

print(*predictions)

```

Test Case 1

Input

```

2 2 2
1 0
0 1
1 0

```

```
0 1
0 0
1 -1
0
1 0
0.1
```

Output

```
0.3133
1.0134 0.0000
0.0000 1.0134
0.0134 0.0134
1.0134 -1.0134
0.0000
0.7363 0.2637
1 0
```

Problem Description 5

Implement a simple image-preprocessing pipeline for a grayscale image using NumPy.

You are given:

- a grayscale image
- a binary `flip` value

If:

```
flip = 1
```

horizontally flip the image so that the leftmost column becomes the rightmost column.

If:

```
flip = 0
```

keep the original image unchanged.

After the optional flip, apply Min-Max normalization over the complete image:

```
normalized =
(image - minimum_value)
/
(maximum_value - minimum_value)
```

The smallest pixel value must therefore become 0, while the largest becomes 1.

If every pixel has the same value, the denominator becomes zero. In this case, return an image containing only zeros.

Input Format

```
H W
flip
image row 1
image row 2
...
image row H
```

Output Format

Print the normalized image.

Every value must be printed to exactly four decimal places.

Constraints

```
1 <= H, W <= 20
flip = 0 or 1
-1000 <= pixel value <= 1000
```

Boilerplate

HEAD

```
import sys
import numpy as np
```

BODY

```
def preprocess_image(image, flip):
    # Write your code here using NumPy
    pass
```

TAIL

```
data = sys.stdin.read().strip().split()
it = iter(data)

H = int(next(it))
W = int(next(it))

flip = int(next(it))

image = np.array(
    [
        [float(next(it)) for _ in range(W)]
        for _ in range(H)
    ],
    dtype=float
)

result = preprocess_image(image, flip)

for row in result:
    print(
        " ".join(
            f"{value:.4f}"
            for value in row
        )
    )
```

Test Case 1

Input

```
2 2
0
0 1
2 3
```

Output

```
0.0000 0.3333
0.6667 1.0000
```

Problem Description 6

Implement the retrieval and context-construction component of a Retrieval-Augmented Generation system.

You are given one query embedding, n document chunks with their embeddings, the maximum number of retrieval candidates k , a maximum permitted embedding distance, and a maximum word budget for the final context.

Measure query-document distance using Euclidean distance:

```
distance = sqrt(
    sum(
        (query[i] - document_embedding[i]) ** 2
    )
)
```

A document is eligible only when:

```
distance <= maximum_distance
```

Rank eligible documents from smallest to largest distance.

If two documents have exactly the same distance, the document having the smaller zero-based original index must appear first.

Only the first k ranked eligible documents are considered for the final context.

Process these candidates in ranked order.

Select a chunk only if adding all of its whitespace-separated words does not exceed `max_words`.

If a chunk does not fit, skip it and continue examining the remaining candidates.

Join selected chunks using:

```
|
```

Output:

- selected document indices
- their Euclidean distances
- final context

If no document is selected, output:

```
NONE
NONE
NO_CONTEXT
```

Input Format

```
n d k

maximum_distance max_words

query_embedding

document_text_0
embedding_0

document_text_1
embedding_1

...

document_text_n-1
embedding_n-1
```

Output Format

If documents are selected:

```
selected_indices
selected_distances
constructed_context
```

Distances must be printed to exactly four decimal places.

If none are selected:

```
NONE
NONE
NO_CONTEXT
```

Constraints

```
1 <= n <= 20
1 <= d <= 20
1 <= k <= n

maximum_distance >= 0

1 <= max_words <= 100

Document text does not contain |.
```

Boilerplate

HEAD

```
import sys
import numpy as np
```

BODY

```
def retrieve_context(
    query,
    documents,
    embeddings,
    k,
    maximum_distance,
    max_words
):
    # Write your code here using NumPy
    pass
```

TAIL

```
n, d, k = map(
    int,
    sys.stdin.readline().split()
)

maximum_distance, max_words = (
    sys.stdin.readline().split()
)

maximum_distance = float(maximum_distance)
max_words = int(max_words)

query = np.array(
    list(
        map(
            float,
            sys.stdin.readline().split()
        )
    ),
    dtype=float
)

documents = []
embeddings = []

for _ in range(n):
    document = sys.stdin.readline().rstrip("\n")

    embedding = list(
        map(
            float,
            sys.stdin.readline().split()
        )
    )

    documents.append(document)
    embeddings.append(embedding)

embeddings = np.array(
    embeddings,
    dtype=float
)

indices, distances, context = retrieve_context(
```

```

        query,
        documents,
        embeddings,
        k,
        maximum_distance,
        max_words
    )

if len(indices) == 0:

    print("NONE")
    print("NONE")
    print("NO_CONTEXT")

else:

    print(*indices)

    print(
        " ".join(
            f"{value:.4f}"
            for value in distances
        )
    )

    print(context)

```

Test Case 1

Input

```

4 2 3
1.1 4
0 0
alpha beta
0 0
gamma
1 0
delta epsilon
0 1
zeta
2 2

```

Output

```

0 1
0.0000 1.0000
alpha beta | gamma

```

Problem Description 7

A binary-classification model produces one raw logit for each of m input samples.

Implement the output-processing stage using the Sigmoid activation function and Binary Cross-Entropy loss.

For every logit z , calculate:

$$p = 1 / (1 + \exp(-z))$$

For binary target y , calculate:

```
loss =  
-(  
    y * ln(p)  
    +  
    (1 - y) * ln(1 - p)  
)
```

Calculate the BCE loss separately for every sample.

The final loss is:

```
mean_loss = mean of all individual BCE losses
```

The predicted class is:

```
1 if p >= 0.5  
0 otherwise
```

The implementation must remain numerically stable for large positive and negative logits.

Input Format

```
m  
  
z1 z2 ... zm  
  
y1 y2 ... ym
```

Where:

- m is the number of samples
- the second line contains model logits
- the third line contains binary target values

Output Format

```
p1 p2 ... pm  
L1 L2 ... Lm  
mean_loss  
c1 c2 ... cm
```

Print all floating-point values to exactly four decimal places.

Constraints

```
1 <= m <= 50  
-1000 <= z[i] <= 1000
```

Each target value is either 0 or 1.

Use the natural logarithm.

A probability exactly equal to 0.5 must be classified as 1.

Boilerplate

HEAD

```
import sys
import numpy as np
```

BODY

```
def sigmoid_bce(logits, targets):
    # Write your code here using NumPy
    pass
```

TAIL

```
data = sys.stdin.read().strip().split()
it = iter(data)

m = int(next(it))

logits = np.array(
    [float(next(it)) for _ in range(m)],
    dtype=float
)

targets = np.array(
    [int(next(it)) for _ in range(m)],
    dtype=int
)

(
    probabilities,
    losses,
    mean_loss,
    predictions
) = sigmoid_bce(
    logits,
    targets
)

print(
    " ".join(
        f"{value:.4f}"
        for value in probabilities
    )
)

print(
    " ".join(
        f"{value:.4f}"
        for value in losses
    )
)

print(f"{mean_loss:.4f}")

print(*predictions)
```

Test Case 1

Input

```
3
0 1 -1
1 1 0
```

Output

```
0.5000 0.7311 0.2689
0.6931 0.3133 0.3133
0.4399
1 1 0
```

Problem Description 8

A Transformer must contain information about the positions of tokens because self-attention alone does not represent sequence order.

Implement sinusoidal positional encoding using NumPy.

You are given:

- sequence length L
- model dimension D

For every position pos and dimension j , first calculate:

```
pair_index = j // 2

angle =
pos /
(10000 raised to power ((2 * pair_index) / D))
```

For an even dimension:

```
PE[pos, j] = sin(angle)
```

For an odd dimension:

```
PE[pos, j] = cos(angle)
```

Positions and dimensions both begin from index 0.

If D is odd, the final even dimension still follows the sine rule.

Input Format

L D

Output Format

Print the positional-encoding matrix containing L rows and D columns.

Print every value to exactly four decimal places.

Constraints

```
1 <= L <= 100
1 <= D <= 100
```

Boilerplate

HEAD

```
import sys
import numpy as np
```

BODY

```
def positional_encoding(
    sequence_length,
    model_dimension
):
    # Write your code here using NumPy
    pass
```

TAIL

```
L, D = map(
    int,
    sys.stdin.read().strip().split()
)

result = positional_encoding(L, D)

for row in result:
    print(
        " ".join(
            f"{value:.4f}"
            for value in row
        )
    )
```

Test Case 1

Input

```
1 4
```

Output

```
0.0000 1.0000 0.0000 1.0000
```

Problem Description 9

Low-Rank Adaptation, or LoRA, adapts a pre-trained model by adding a low-rank update to an existing weight matrix instead of updating all base-model parameters.

Implement a LoRA-adapted linear layer using NumPy.

You are given:

- an input matrix X
- a frozen base weight matrix W
- two low-rank matrices A and B
- scaling value α
- rank r

Calculate the low-rank update as:

```
low_rank_update = A @ B
```

Scale the update using:

```
scaled_update =  
(alpha / r) * low_rank_update
```

The adapted weight matrix is:

```
adapted_weight =  
W + scaled_update
```

Calculate the final output as:

```
output =  
X @ adapted_weight
```

No bias is used.

Do not modify the original base weight matrix directly.

Input Format

```
m input_dim output_dim r  
alpha
```

```
X row 1  
...  
X row m
```

```
W row 1  
...  
W row input_dim
```

```
A row 1  
...  
A row input_dim
```

```
B row 1  
...  
B row r
```

Where:

X has dimensions $m \times \text{input_dim}$

W has dimensions $\text{input_dim} \times \text{output_dim}$

A has dimensions $\text{input_dim} \times r$

B has dimensions $r \times \text{output_dim}$

Output Format

Print the final output matrix.

Every value must be printed to exactly four decimal places.

Constraints

```
1 <= m <= 20
1 <= input_dim <= 20
1 <= output_dim <= 20
1 <= r <= min(input_dim, output_dim)
0 <= alpha <= 100
```

Boilerplate

HEAD

```
import sys
import numpy as np
```

BODY

```
def lora_forward(
    X,
    W,
    A,
    B,
    alpha
):
    # Write your code here using NumPy
    pass
```

TAIL

```
data = sys.stdin.read().strip().split()
it = iter(data)

m = int(next(it))
input_dim = int(next(it))
output_dim = int(next(it))
r = int(next(it))

alpha = float(next(it))

X = np.array(
    [
        [
            float(next(it))
            for _ in range(input_dim)
        ]
        for _ in range(m)
    ]
)

W = np.array(
    [
        [
            float(next(it))
            for _ in range(output_dim)
        ]
        for _ in range(input_dim)
    ]
)

A = np.array(
    [
        [
            float(next(it))
```

```

        for _ in range(r)
    ]
    for _ in range(input_dim)
]
)

B = np.array(
    [
        [
            float(next(it))
            for _ in range(output_dim)
        ]
        for _ in range(r)
    ]
)

result = lora_forward(
    X,
    W,
    A,
    B,
    alpha
)

for row in result:
    print(
        " ".join(
            f"{value:.4f}"
            for value in row
        )
    )

```

Test Case 1

Input

```

1 2 2 1
1
1 2
1 0
0 1
1
0
1 1

```

Output

```

2.0000 3.0000

```

Problem Description 10

Implement the retrieval and context-construction component of a Retrieval-Augmented Generation system using embedding dot-product relevance.

You are given:

- one query embedding
- n document chunks with their embeddings
- maximum number of retrieval candidates k

- minimum permitted relevance score
- word budget for the final context

Calculate document relevance using:

```
score = query @ document_embedding
```

A document is eligible only when:

```
score >= minimum_score
```

Rank eligible documents from highest to lowest score.

If two documents have exactly the same score, the document with the smaller original zero-based index must rank first.

Only the first k ranked eligible documents may be considered for the final context.

Process those candidates in ranked order.

A chunk may be selected only when all its whitespace-separated words fit within the remaining `max_words` budget.

If a chunk does not fit, skip it and continue examining the remaining candidates.

Join selected document chunks using:

```
|
```

Output:

- selected document indices
- their relevance scores
- resulting context

If no document is selected, print exactly:

```
NONE
NONE
NO_CONTEXT
```

Input Format

```
n d k
```

```
minimum_score max_words
```

```
query_embedding
```

```
document_text_0
embedding_0
```

```
document_text_1
embedding_1
```

```
...
```

```
document_text_n-1
```

embedding_n-1

Output Format

When documents are selected:

```
selected_indices
selected_scores
constructed_context
```

Print scores to exactly four decimal places.

If no document is selected:

```
NONE
NONE
NO_CONTEXT
```

Constraints

```
1 <= n <= 20
1 <= d <= 20
1 <= k <= n
1 <= max_words <= 100
```

Document text will not contain |.

Boilerplate

HEAD

```
import sys
import numpy as np
```

BODY

```
def retrieve_context(
    query,
    documents,
    embeddings,
    k,
    minimum_score,
    max_words
):
    # Write your code here using NumPy
    pass
```

TAIL

```
n, d, k = map(
    int,
    sys.stdin.readline().split()
)

minimum_score, max_words = (
    sys.stdin.readline().split()
)

minimum_score = float(minimum_score)
```

```

max_words = int(max_words)

query = np.array(
    list(
        map(
            float,
            sys.stdin.readline().split()
        )
    ),
    dtype=float
)

documents = []
embeddings = []

for _ in range(n):
    document = sys.stdin.readline().rstrip("\n")

    embedding = list(
        map(
            float,
            sys.stdin.readline().split()
        )
    )

    documents.append(document)
    embeddings.append(embedding)

embeddings = np.array(
    embeddings,
    dtype=float
)

indices, scores, context = retrieve_context(
    query,
    documents,
    embeddings,
    k,
    minimum_score,
    max_words
)

if len(indices) == 0:
    print("NONE")
    print("NONE")
    print("NO_CONTEXT")

else:
    print(*indices)

    print(
        " ".join(
            f"{value:.4f}"
            for value in scores
        )
    )

    print(context)

```

Test Case 1

Input

```
4 2 3
0.5 4
1 0
alpha beta
1 0
gamma
0.8 0.2
delta epsilon
0 1
zeta
1 0
```

Output

```
0 3 1
1.0000 1.0000 0.8000
alpha beta | zeta | gamma
```

Problem Description 11

A Vision Transformer converts an image into a sequence of patch embeddings before the sequence is processed by Transformer blocks.

Implement the patch-embedding stage of a simplified Vision Transformer using NumPy.

You are given:

- a grayscale image of size $H \times W$
- patch size P
- embedding dimension D
- a patch-projection matrix
- one class-token vector
- positional embeddings

Both H and W are divisible by P .

Divide the image into non-overlapping $P \times P$ patches.

Patches must be extracted:

```
left to right
then top to bottom
```

Flatten every patch in row-major order.

Each flattened patch contains:

```
 $P * P$  values
```

Project the patches using:

```
patch_embeddings =
flattened_patches @ projection_matrix
```

Prepend the supplied class token:

```
tokens =  
class_token followed by patch_embeddings
```

Finally, add the corresponding positional embedding to every token:

```
final_tokens =  
tokens + positional_embeddings
```

Input Format

H W P D

```
image row 1  
...  
image row H
```

```
projection row 1  
...  
projection row P*P
```

class_token

```
positional_embedding row 1  
...  
positional_embedding row number_of_tokens
```

The number of image patches is:

```
number_of_patches =  
(H // P) * (W // P)
```

Therefore:

```
number_of_tokens =  
number_of_patches + 1
```

Output Format

First print:

```
number_of_tokens D
```

Then print the final token matrix.

All values must be printed to exactly four decimal places.

Constraints

```
1 <= H, W <= 20  
1 <= P <= min(H, W)  
H % P = 0  
W % P = 0  
1 <= D <= 20
```

Boilerplate

HEAD

```
import sys
```

```
import numpy as np
```

BODY

```
def create_vit_tokens(
    image,
    patch_size,
    projection,
    class_token,
    positional_embeddings
):
    # Write your code here using NumPy
    pass
```

TAIL

```
data = sys.stdin.read().strip().split()
it = iter(data)

H = int(next(it))
W = int(next(it))
P = int(next(it))
D = int(next(it))

image = np.array(
    [
        [float(next(it)) for _ in range(W)]
        for _ in range(H)
    ]
)

projection = np.array(
    [
        [
            float(next(it))
            for _ in range(D)
        ]
        for _ in range(P * P)
    ]
)

class_token = np.array(
    [float(next(it)) for _ in range(D)]
)

number_of_patches = (
    (H // P) * (W // P)
)

number_of_tokens = (
    number_of_patches + 1
)

positional_embeddings = np.array(
    [
        [
            float(next(it))
            for _ in range(D)
        ]
        for _ in range(number_of_tokens)
    ]
)

result = create_vit_tokens(
    image,
```

```

        P,
        projection,
        class_token,
        positional_embeddings
    )

print(result.shape[0], result.shape[1])

for row in result:

    print(
        " ".join(
            f"{value:.4f}"
            for value in row
        )
    )

```

Test Case 1

Input

```

2 2 1 2
1 2
3 4
1 2
0 0
0 0
0 0
0 0
0 0
0 0
0 0

```

Output

```

5 2
0.0000 0.0000
1.0000 2.0000
2.0000 4.0000
3.0000 6.0000
4.0000 8.0000

```

Problem Description 12

A Transformer encoder applies a position-wise feed-forward network after its attention operation.

Implement a simplified Transformer feed-forward block using NumPy.

You are given:

- input sequence matrix x
- first weight matrix w_1
- first bias vector b_1
- second weight matrix w_2
- second bias vector b_2

First calculate the hidden representation:

```
hidden =  
X @ W1 + b1
```

Apply ReLU activation:

```
hidden =  
maximum(0, hidden)
```

Calculate the feed-forward output:

```
ffn_output =  
hidden @ W2 + b2
```

Add the residual connection:

```
residual =  
X + ffn_output
```

Finally, apply Layer Normalization independently to each token.

For every row:

```
row_mean =  
mean(residual_row)
```

Calculate:

```
row_variance =  
mean(  
    (residual_row - row_mean)  
    raised to power 2  
)
```

Then normalize:

```
normalized =  
(residual_row - row_mean)  
/  
sqrt(row_variance + 0.00001)
```

Do not apply trainable Layer-Normalization scale or bias parameters.

Input Format

L D F

X row 1

...

X row L

W1 row 1

...

W1 row D

b1

W2 row 1

...

W2 row F

b2

Where:

L = sequence length
D = model dimension
F = hidden dimension of the feed-forward network

Matrix dimensions are:

X = L x D
W1 = D x F
b1 = F
W2 = F x D
b2 = D

Output Format

Print the final normalized L x D matrix.

Print every value to exactly four decimal places.

Constraints

1 <= L <= 20
1 <= D <= 20
1 <= F <= 40
Matrix values may be integers or decimal values.

Boilerplate

HEAD

```
import sys
import numpy as np
```

BODY

```
def transformer_ffn(
    X,
    W1,
    b1,
    W2,
    b2
):
    # Write your code here using NumPy
    pass
```

TAIL

```
data = sys.stdin.read().strip().split()
it = iter(data)

L = int(next(it))
D = int(next(it))
F = int(next(it))

X = np.array(
    [
        [float(next(it)) for _ in range(D)]
        for _ in range(L)
    ]
)
```

```

)

W1 = np.array(
    [
        [float(next(it)) for _ in range(F)]
        for _ in range(D)
    ]
)

b1 = np.array(
    [float(next(it)) for _ in range(F)]
)

W2 = np.array(
    [
        [float(next(it)) for _ in range(D)]
        for _ in range(F)
    ]
)

b2 = np.array(
    [float(next(it)) for _ in range(D)]
)

result = transformer_ffn(
    X,
    W1,
    b1,
    W2,
    b2
)

for row in result:

    print(
        " ".join(
            f"{value:.4f}"
            for value in row
        )
    )

```

Test Case 1

Input

```

2 2 2
1 0
0 1
1 0
0 1
0 0
1 0
0 1
0 0

```

Output

```

1.0000 -1.0000
-1.0000 1.0000

```